

Model Binding Using FromRoute in ASP.NET Core Web API

Back to: [ASP.NET Core Web API Tutorials](#)

Model Binding Using FromRoute in ASP.NET Core Web API

In this article, I will discuss **Model Binding Using FromRoute in ASP.NET Core Web API** Application with Examples. Please read our previous article discussing Model Binding Using [FromQuery in ASP.NET Core Web API](#) with Examples.

What is FromRoute Attribute in ASP.NET Core Web API?

The FromRoute attribute is used to specify that a parameter in an action method should be bound from the route data in the URL. It tells the ASP.NET Core model binder to get the value for the parameter from the route parameters rather than the query string or request body. When we define routes using placeholders (like {id} in the URL path), the FromRoute attribute allows us to capture those placeholder values and pass them as parameters to controller actions.

How Does the FromRoute Attribute Work in ASP.NET Core Web API?

When an action method parameter is decorated with the FromRoute attribute, ASP.NET Core's model binding system attempts to extract the value from the route data (which is part of the URL) and bind it to the corresponding parameter in the method. For example, given a route template like api/products/{id}, the id segment in the URL can be captured using a parameter decorated with [FromRoute]:

```
[HttpGet("api/products/{id}")]
public IActionResult GetProduct([FromRoute] int id)
{
    // id will be populated with the value from the route
}
```

Example to Understand FromRoute Attribute in ASP.NET Core Web API:

Let us see an example of Understanding the FromRoute attribute in ASP.NET Core Web API. We are going to use the following Product model. So, create a class file named **Product.cs** and then copy and paste the following code:

```
namespace ModelBinding.Models
{
    public class Product
    {
```

```

        public int Id { get; set; }
        public string Name { get; set; }
        public int Quantity { get; set; }
        public string Category { get; set; }
        public int Price { get; set; }
    }
}

```

Using FromRoute Attribute in ASP.NET Core Web API:

Suppose you have an API endpoint that retrieves a specific Product by ID. The ID is part of the URL path. For a better understanding, please create an API Controller with the name Products Controller and then copy and paste the following code:

```

using Microsoft.AspNetCore.Mvc;
using ModelBinding.Models;

namespace ModelBinding.Controllers
{
    [Route("api/[controller]")]
    [ApiController]
    public class ProductsController : ControllerBase
    {
        private static List<Product> Products = new List<Product>
        {
            new Product { Id = 1, Name = "Laptop", Category = "Electronics",
Price = 1000, Quantity = 10 },
            new Product { Id = 2, Name = "Desktop", Category = "Electronics",
Price = 2000, Quantity = 20 },
            new Product { Id = 3, Name = "Mobile", Category = "Electronics",
Price = 3000, Quantity = 30 },
            new Product { Id = 4, Name = "Casual Shirts", Category = "Apparel",
Price = 500, Quantity = 10 },
            new Product { Id = 5, Name = "Formal Shirts", Category = "Apparel",
Price = 600, Quantity = 30 },
            new Product { Id = 6, Name = "Jackets & Coats", Category = "Apparel",
Price = 700, Quantity = 20 },
        };

        [HttpGet("{id}")]
        public IActionResult GetProductById([FromRoute] int id)
        {
            // Logic to retrieve the user by ID
            var product = Products.FirstOrDefault(prd => prd.Id == id);
            if (product != null)
            {

```

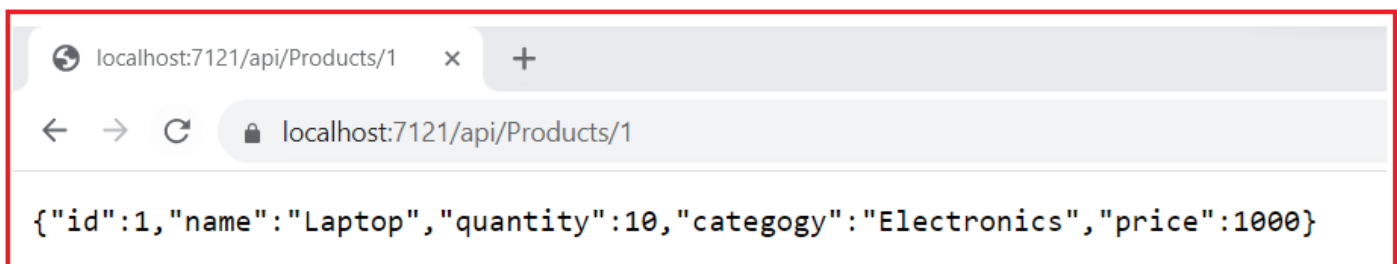
```

        return Ok(product);
    }
    return NotFound($"No Product Found with Product Id: {id}");
}
}
}

```

In this example, the `[HttpGet("{id}")]` attribute specifies that this action handles GET requests where the URL path includes an id value (e.g., `/api/products/1`). The id parameter in the `GetProductById` method is decorated with `[FromRoute]`, telling ASP.NET Core to bind the value from the route data to this parameter.

When a request is made to `/api/products/1`, ASP.NET Core matches the request to the `GetProductById` action because the URL path fits the route template defined by `[HttpGet("{id}")]`. The framework extracts the "1" part of the URL and assigns it to the id parameter of the `GetProduct` method because the id is specified as a route parameter and is marked with `[FromRoute]`. Now, run the application and access the above endpoint as follows:



How to Assign a Different Name to Route Parameter in ASP.NET Core Web API

The `Name` property of the `FromRoute` attribute is used when the parameter name in the action method does not match the placeholder name in the route template. By specifying the `Name` property, we can explicitly map a route placeholder to a differently named parameter in the method. This can be useful when the parameter name in the action method is different from the route segment name. For a better understanding, please modify the `Products Controller` as follows. The following code is self-explained, so please go through the comment lines.

```

using Microsoft.AspNetCore.Mvc;
using ModelBinding.Models;

namespace ModelBinding.Controllers
{
    [Route("api/[controller]")]
    [ApiController]
    public class ProductsController : ControllerBase
    {
        private static List<Product> Products = new List<Product>
        {
            new Product { Id = 1, Name = "Laptop", Category = "Electronics",
                Price = 1000, Quantity = 10 },

```

```

        new Product { Id = 2, Name = "Desktop", Category = "Electronics",
Price = 2000, Quantity = 20 },
        new Product { Id = 3, Name = "Mobile", Category = "Electronics",
Price = 3000, Quantity = 30 },
        new Product { Id = 4, Name = "Casual Shirts", Category = "Apparel",
Price = 500, Quantity = 10 },
        new Product { Id = 5, Name = "Formal Shirts", Category = "Apparel",
Price = 600, Quantity = 30 },
        new Product { Id = 6, Name = "Jackets & Coats", Category = "Apparel",
Price = 700, Quantity = 20 },
    };

    // Assuming your route template specifies a parameter named "Id"
    // but you want to use "ProductId" within your action method
    [HttpGet("{Id}")]
    public IActionResult GetProductById([FromRoute(Name = "Id")] int
ProductId)
    {
        // Now, you can use "ProductId" to refer to the parameter that comes
from the route.
        // Fetch the Product by the ProductId and return a response

        var product = Products.FirstOrDefault(prd => prd.Id == ProductId);
        if (product != null)
        {
            return Ok(product);
        }
        return NotFound($"No Product Found with Product Id: {ProductId}");
    }
}

```

In this example:

- The route defined in the `HttpGet` attribute includes a parameter named `Id`.
- In the `GetProductById` method signature, the `FromRoute` attribute binds the route's `Id` value to the method parameter `ProductId`.
- Inside the method, you can refer to the parameter using the name `ProductId`, even though the route specifies the name as `Id`.

FromRoute Attribute with Complex Type in ASP.NET Core Web API:

While the `FromRoute` attribute is generally used with simple types like `int`, `string`, etc., you can also use it with complex types if your route segments map directly to properties of the complex type. This is less common but can

be useful in scenarios where you have a structured route and want to bind it directly to an object. Let us understand this with an example. First, create a class file named **ProductRoute.cs** and then copy and paste the following code:

```
namespace ModelBinding.Models
{
    public class ProductRoute
    {
        public string Name { get; set; }
        public string Category { get; set; }
    }
}
```

Modifying the Products Controller:

Next, modify the Products Controller as follows. Here, we have marked the **ProductRoute** parameter with the **FromRoute** Attribute, meaning this parameter's properties will map with the Route Data parameter.

```
using Microsoft.AspNetCore.Mvc;
using ModelBinding.Models;

namespace ModelBinding.Controllers
{
    [Route("api/[controller]")]
    [ApiController]
    public class ProductsController : ControllerBase
    {
        private static List<Product> Products = new List<Product>
        {
            new Product { Id = 1, Name = "Laptop", Category = "Electronics",
                Price = 1000, Quantity = 10 },
            new Product { Id = 2, Name = "Desktop", Category = "Electronics",
                Price = 2000, Quantity = 20 },
            new Product { Id = 3, Name = "Mobile", Category = "Electronics",
                Price = 3000, Quantity = 30 },
            new Product { Id = 4, Name = "Casual Shirts", Category = "Apparel",
                Price = 500, Quantity = 10 },
            new Product { Id = 5, Name = "Casual Pants", Category = "Apparel",
                Price = 600, Quantity = 30 },
            new Product { Id = 6, Name = "Jackets & Coats", Category = "Apparel",
                Price = 700, Quantity = 20 },
        };

        [HttpGet("{Name}/{Category}")]
        public IActionResult GetProductById([FromRoute] ProductRoute
            productRoute)
        {
            // Implementation logic here
        }
    }
}
```

```

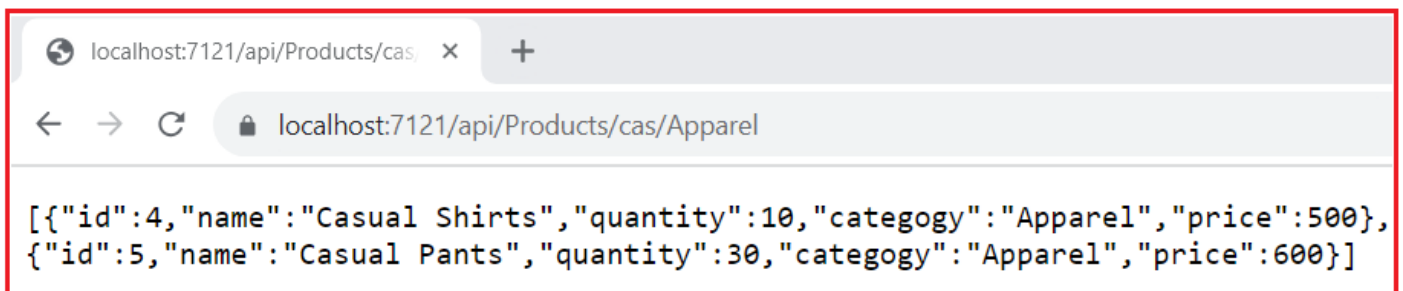
        if(productRoute != null)
        {
            var FilteredProducts = Products.Where(prd =>
prd.Name.ToLower().StartsWith(productRoute.Name) &&
            prd.Category.ToLower() ==
productRoute?.Category.ToLower()).ToList();

            return Ok(FilteredProducts);
        }

        return NotFound($"No Product Found");
    }
}
}

```

With the above changes in place, run the application and access the above endpoint as follows, and you should get the output as expected.



When Should We Use FromRoute Attribute in ASP.NET Core Web API?

We need to use the FromRoute attribute when we need to extract and use specific segments of the URL in action method parameters. It is particularly useful when:

- Retrieving resource identifiers (like user IDs or product IDs) that are part of the URL.
- Implementing RESTful routes where the URL contains significant parameters for action determination.

In the next article, I will discuss [Model Binding Using FromHeader in ASP.NET Core Web API](#) with Examples. In this article, I try to explain **Model Binding Using FromRoute in ASP.NET Core Web API** with Examples, and I hope you enjoy this article “Model Binding Using FromRoute in ASP.NET Core Web API”.



Dot Net Tutorials

About the Author: Pranaya Rout

Pranaya Rout has published more than 3,000 articles in his 11-year career. Pranaya Rout has very good experience with Microsoft Technologies, Including C#, VB, ASP.NET MVC, ASP.NET

Web API, EF, EF Core, ADO.NET, LINQ, SQL Server, MYSQL, Oracle, ASP.NET Core, Cloud Computing, Microservices, Design Patterns and still learning new technologies.

1 thought on “Model Binding Using FromRoute in ASP.NET Core Web API”



SAMEERA

MARCH 11, 2025 AT 4:00 PM

Thanks sir

[Reply](#)

Privacy Preferences

We and our partners share information on your use of this website to help improve your experience. For more information, or to opt out click the Do Not Sell My Information button below.

Consent

Do Not Sell My Information